Name: Sanskruti Dahiphale
PRN: 123B1B007

Assignment No. 5

Title:
You are given a set of items, each with a weight and profit, and a maximum weight capacity. Using the Branch and Bound approach, find the optimal subset of items that maximizes total profit while staying within the weight limit. Implement and compare FIFO, LIFO, and Least Cost (LC) approaches for this problem.

Objectives:
- To understand State Space Trees and bounding functions.
- To compare different Branch and Bound search strategies.

Course Outcome Mapping:
CO3 - Apply computational techniques to solve optimization and decision-making problems using either Dynamic programming or Branch and Bound.

Theory:

Branch and Bound Strategy:
Branch and Bound is an efficient optimization technique used to solve problems where the goal is to find the best solution among many possible candidates, such as in the 0/1 Knapsack problem.

The method works by exploring solutions in a structured manner using a State Space Tree, where each node represents a partial decision or solution. From each node, the algorithm generates further possibilities by making choices, typically deciding whether to include or exclude a particular element.

To avoid exploring all possible combinations (which would be very time-consuming), the algorithm uses a bounding function. This function estimates the best possible outcome that can be achieved from a given node. If this estimated value is worse than the best solution already found, that branch is not explored further.

By eliminating such non-promising paths early (a process called pruning), Branch and Bound significantly reduces the number of computations and improves overall efficiency while still

guaranteeing an optimal solution.

Upper Bound Calculation (0/1 Knapsack):
In the Branch and Bound approach for the 0/1 Knapsack problem, the **upper bound** represents the maximum possible profit that can be achieved from a given node.

To compute this bound, the idea of the **fractional knapsack** is used as an approximation:

- Items are selected in order of highest profit-to-weight ratio.
- Full items are added to the knapsack as long as there is available capacity.
- If the remaining capacity is insufficient for the next item, only a proportional (fractional) part of that item is considered.

This method provides an optimistic estimate of the best possible profit from that node. If this estimated value is lower than the current best solution, the node is discarded, helping to reduce unnecessary exploration..

Search Strategies:

1. FIFO (First In First Out):

• Uses a queue (Breadth-First Search).
• Nodes are explored level by level.
• May explore more nodes, leading to higher computation time.

2. LIFO (Last In First Out):

• Uses a stack (Depth-First Search).
• Explores one branch deeply before backtracking.
• May reach a solution quickly but not always efficient.

3. Least Cost (LC) / Best First Search:

• Uses a priority queue.
• Selects node with the highest bound (maximum potential profit). •
More efficient as it explores the most promising nodes first.

Feature	**FIFO**	**LIFO**	**Least Cost (LC)**
Data Structure	Queue	Stack	Priority Queue
Traversal	Level-wise (BFS)	Depth-first (DFS)	Best-first
Node Selection	Order of insertion	Last inserted	Highest bound first
Efficiency	Moderate	Moderate	High

Example:

Consider the following items:

Item	Weight	
1	20	10
2	12	45
3	24	45
4	5	13

Maximum Capacity = 50

Step 1: Calculate Profit/Weight Ratio

	Ratio
2	3.75
4	2.60
3	1.88
1	0.50

Step 2: State Space Tree

- Root node represents no items selected
- Each level represents a decision (include/exclude item)
- Branching generates possible combinations

Step 3: Apply Bounding

- Calculate upper bound for each node
- If bound < current best profit → prune node

Step 4: Optimal Solution

Best combination:

- Item 2 + Item 3 + Item 4

Total:

- Weight = 41
- Profit = 103

Observation:

- All approaches give the same optimal profit

- FIFO explores more nodes
- LIFO explores depth-wise
- LC is more efficient as it selects best nodes first

Algorithm:

Branch and Bound Algorithm Procedure (0/1 Knapsack):

1. Start by creating the initial (root) node with level = -1, profit = 0, and weight = 0.
2. Compute the upper bound for this node to estimate the maximum possible profit.
3. Insert the node into an appropriate data structure such as a queue, stack, or priority queue (commonly a priority queue for best-first search).
4. Repeat the following steps until the structure becomes empty:
    - Select and remove a node from the structure.
    - Check if the node is promising (i.e., its bound is greater than the current maximum profit).
    - If promising, generate its child nodes by considering inclusion and exclusion of the next item.
    - Calculate the bound for each child node.
    - Update the maximum profit if a better feasible solution is found.
    - Insert only the promising child nodes back into the structure for further exploration.
5. Continue this process until all nodes are either explored or pruned, ensuring the optimal solution is obtained efficiently.

Time Complexity :

- **Worst Case:** $O(2^n)$
- Each item presents two possible decisions: either include it in the knapsack or exclude it.
- As a result, the total number of possible combinations (subsets) is 2^n.

However, in practical scenarios, the use of a bounding function helps eliminate non-promising branches early. This pruning significantly reduces the number of nodes explored, making the algorithm much more efficient than the brute-force approach in most cases.

Code:

```
from queue import Queue
import heapq
# Item class
class Item:
    def __init__(self, weight, profit):
        self.weight = weight
        self.profit = profit
```

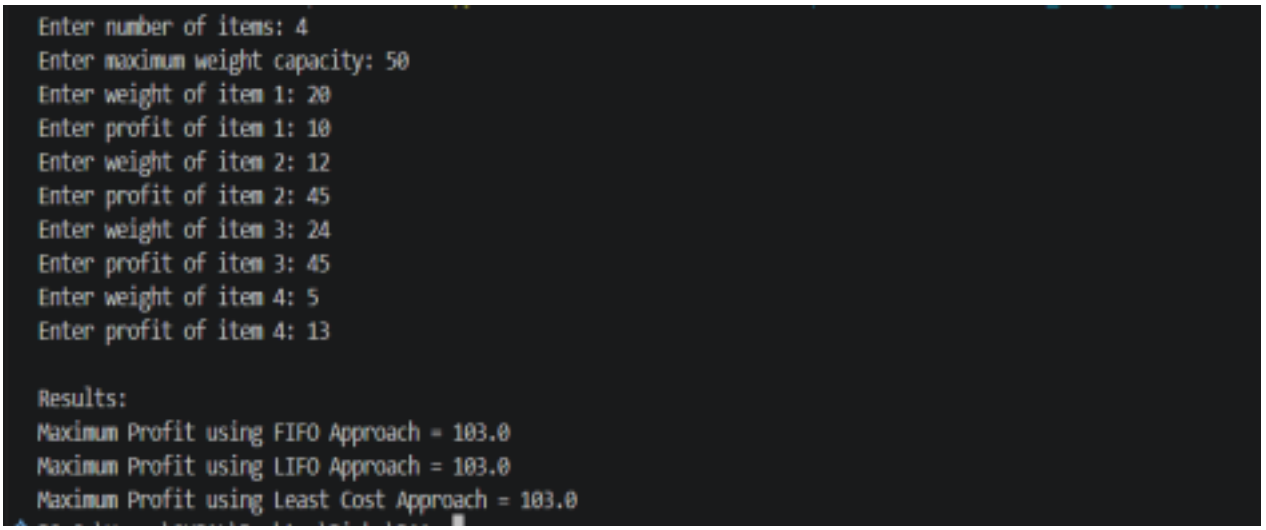
```
        self.ratio = profit / weight
# Node class
class Node:
    def __init__(self, level, profit, weight):
        self.level = level
        self.profit = profit
        self.weight = weight
        self.bound = 0

# Bound function
def bound(node, n, W, items):
    if node.weight >= W:
        return 0
    profit_bound = node.profit
    j = node.level + 1
    total_weight = node.weight
    while j < n and total_weight + items[j].weight <= W:
        total_weight += items[j].weight
        profit_bound += items[j].profit
        j += 1
    if j < n:
        profit_bound += (W - total_weight) * items[j].ratio
    return profit_bound

# FIFO Approach (Queue)
def knapsack_fifo(W, items, n):
    q = Queue()
    u = Node(-1, 0, 0)
    u.bound = bound(u, n, W, items)
    q.put(u)
    max_profit = 0
    while not q.empty():
        u = q.get()
        if u.level == n - 1:
```

```
            continue
        v = Node(u.level + 1, 0, 0)

        # Include item
        v.weight = u.weight + items[v.level].weight
        v.profit = u.profit + items[v.level].profit if
        v.weight <= W and v.profit > max_profit:
        max_profit = v.profit
        v.bound = bound(v, n, W, items)
        if v.bound > max_profit:
            q.put(v)

        # Exclude item
        v = Node(u.level + 1, u.profit, u.weight)
        v.bound = bound(v, n, W, items)

        if v.bound > max_profit:
            q.put(v)
    return max_profit

# LIFO Approach (Stack)
def knapsack_lifo(W, items, n):
    stack = []
    u = Node(-1, 0, 0)
    u.bound = bound(u, n, W, items)
    stack.append(u)
    max_profit = 0
    while stack:
        u = stack.pop()
        if u.level == n - 1:
            continue
        v = Node(u.level + 1, 0, 0)

        # Include item
```

```
        v.weight = u.weight + items[v.level].weight
        v.profit = u.profit + items[v.level].profit if
        v.weight <= W and v.profit > max_profit:
        max_profit = v.profit
        v.bound = bound(v, n, W, items)
        if v.bound > max_profit:
            stack.append(v)

        # Exclude item
        v = Node(u.level + 1, u.profit, u.weight)
        v.bound = bound(v, n, W, items)
        if v.bound > max_profit:
            stack.append(v)
    return max_profit

# Least Cost Approach (Priority
Queue) def knapsack_lc(W, items, n):
    pq = []
    u = Node(-1, 0, 0)
    u.bound = bound(u, n, W, items)
    heapq.heappush(pq, (-u.bound, u))
    max_profit = 0
    while pq:
        _, u = heapq.heappop(pq)
        if u.bound <= max_profit:
            continue
        v = Node(u.level + 1, 0, 0)

        # Include item
        v.weight = u.weight + items[v.level].weight
        v.profit = u.profit + items[v.level].profit
        if v.weight <= W and v.profit > max_profit:
            max_profit = v.profit
        v.bound = bound(v, n, W, items)
```

```
        if v.bound > max_profit:
            heapq.heappush(pq, (-v.bound, v))

        # Exclude item
        v = Node(u.level + 1, u.profit, u.weight)
        v.bound = bound(v, n, W, items)
        if v.bound > max_profit:
            heapq.heappush(pq, (-v.bound, v))
    return max_profit

# -------- MAIN --------
n = int(input("Enter number of items: "))
W = float(input("Enter maximum weight capacity: "))
items = []
for i in range(n):
    w = float(input(f"Enter weight of item {i+1}: "))
    p = float(input(f"Enter profit of item {i+1}: "))
    items.append(Item(w, p))

# Sort items by profit/weight ratio
items.sort(key=lambda x: x.ratio, reverse=True)

# Run all approaches
fifo_result = knapsack_fifo(W, items, n)
lifo_result = knapsack_lifo(W, items, n)
lc_result = knapsack_lc(W, items, n)

# Output
print("\nResults:")
print("Maximum Profit using FIFO Approach =", fifo_result)
print("Maximum Profit using LIFO Approach =", lifo_result)
print("Maximum Profit using Least Cost Approach =", lc_result)
```

Output:



Conclusion:

The Branch and Bound technique provides an effective way to solve the 0/1 Knapsack problem by minimizing unnecessary computations through the use of pruning. It systematically explores possible solutions while discarding non-promising paths using bounding.

Among the different strategies, the Least Cost (Best-First Search) approach is the most efficient, as it prioritizes nodes with the highest potential profit, leading to faster convergence and fewer node explorations. In contrast, FIFO (Breadth-First) and LIFO (Depth-First) approaches are easier to implement but may result in exploring a larger number of nodes.

Therefore, using Branch and Bound with the Least Cost strategy ensures both optimality and improved efficiency in solving the knapsack problem.